DISEASE RISK PREDICTION

Instructor:
Dr/ Wael Karam

Presented by:

Mariam Ramadan

Malak Maher

Sandra Waheed

Sham Mohamed

AGENDA

Introduction

Dataset overview

Data preprocessing

-Handling missing values and duplicates

-Handling outliers

-Feature engineering

-Feature selection

Data visualization

Correlation

Algorithms

INTRODUCTION

This project focuses on building a machine learning model that predicts an individual's risk of developing disease based on a comprehensive Health and Lifestyle dataset. The dataset contains essential information related to personal demographics, lifestyle patterns, and key health indicators. These factors are widely recognized in medical research as strong predictors of disease development and overall health outcomes. The dataset includes variables such as:

-Age and Gender
-Biometric measurements (BMI, systolic_bp, diastolic_bp, cholesterol)
-Daily-life factors such as sleep hours, daily steps and water intake

By analyzing these variables and training predictive models, this project aims to support early disease-risk identification and contribute to more proactive health management strategies. Predictive analytics in healthcare has been shown to improve prevention, help identify high-risk individuals, and guide effective interventions when used responsibly and ethically.

DATASET OVERVIEW

head() to visualize first 5 rows
tail() to visualize last 5 rows

```
import numpy as np
import pandas as pd
import seaborn as sb
import matplotlib.pyplot as plt

health_data = pd.read_csv('health_lifestyle_dataset.csv')
health_data.head()
```

	id	age	gender	bmi	daily_steps	sleep_hours	water_intake_l	calories_consumed	smoker	alcohol	resting_hr	systolic_bp	diastolic_bp	cholesterol	family_history	disease_risk
0	1	56	Male	20.5	4198	3.9	3.4	1602	0	0	97	161	111	240	0	0
1	2	69	Female	33.3	14359	9.0	4.7	2346	0	1	68	116	65	207	0	0
2	3	46	Male	31.6	1817	6.6	4.2	1643	0	1	90	123	99	296	0	0
3	4	32	Female	38.2	15772	3.6	2.0	2460	0	0	71	165	95	175	0	0
4	5	60	Female	33.6	6037	3.8	4.0	3756	0	1	98	139	61	294	0	0

```
health_data.tail()
```

	id	age	gender	bmi	daily_steps	sleep_hours	water_intake_l	calories_consumed	smoker	alcohol	resting_hr	systolic_bp	diastolic_bp	cholesterol	family_history	disease_risk
99995	99996	53	Male	33.1	4726	3.9	2.0	3118	0	1	56	105	76	282	0	0
99996	99997	22	Male	35.1	11554	4.5	3.1	1967	0	0	51	149	77	192	0	0
99997	99998	37	Male	18.9	3924	3.8	1.0	2328	0	0	69	92	117	218	0	0
99998	99999	72	Female	27.8	16110	5.6	0.8	3093	0	0	93	164	72	188	0	0
99999	100000	37	Male	35.4	8222	9.1	1.8	3942	0	1	71	145	80	276	0	1

Gives a concise summary of the Data Frame's structure.

```
health_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 16 columns):
 #   Column             Non-Null Count   Dtype
---  ------             --------------   -----
 0   id                 100000 non-null  int64
 1   age                100000 non-null  int64
 2   gender             100000 non-null  object
 3   bmi                100000 non-null  float64
 4   daily_steps        100000 non-null  int64
 5   sleep_hours        100000 non-null  float64
 6   water_intake_l     100000 non-null  float64
 7   calories_consumed  100000 non-null  int64
 8   smoker             100000 non-null  int64
 9   alcohol            100000 non-null  int64
 10  resting_hr         100000 non-null  int64
 11  systolic_bp        100000 non-null  int64
 12  diastolic_bp       100000 non-null  int64
 13  cholesterol        100000 non-null  int64
 14  family_history     100000 non-null  int64
 15  disease_risk       100000 non-null  int64
dtypes: float64(3), int64(12), object(1)
memory usage: 12.2+ MB
```

Provides summary statistics of the data.

```
health_data.describe()
```

	id	age	bmi	daily_steps	sleep_hours	water_intake_l	calories_consumed	smoker	alcohol	resting_hr	systolic_bp	diastolic_bp	cholesterol	family_history	disease_risk
count	100000.000000	100000.000000	100000.000000	100000.00000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.00000	100000.000000	100000.000000	100000.000000	100000.000000
mean	50000.500000	48.525990	29.024790	10479.87029	6.491784	2.751496	2603.341200	0.200940	0.300020	74.457420	134.58063	89.508850	224.300630	0.299150	0.248210
std	28867.657797	17.886768	6.352666	5483.63236	2.021922	1.297338	807.288563	0.400705	0.458269	14.423715	25.95153	17.347041	43.327749	0.457888	0.431976
min	1.000000	18.000000	18.000000	1000.00000	3.000000	0.500000	1200.000000	0.000000	0.000000	50.000000	90.00000	60.000000	150.000000	0.000000	0.000000
25%	25000.750000	33.000000	23.500000	5729.00000	4.700000	1.600000	1906.000000	0.000000	0.000000	62.000000	112.00000	74.000000	187.000000	0.000000	0.000000
50%	50000.500000	48.000000	29.000000	10468.00000	6.500000	2.800000	2603.000000	0.000000	0.000000	74.000000	135.00000	89.000000	224.000000	0.000000	0.000000
75%	75000.250000	64.000000	34.500000	15229.00000	8.200000	3.900000	3299.000000	0.000000	1.000000	87.000000	157.00000	105.000000	262.000000	1.000000	0.000000
max	100000.000000	79.000000	40.000000	19999.00000	10.000000	5.000000	3999.000000	1.000000	1.000000	99.000000	179.00000	119.000000	299.000000	1.000000	1.000000

DATA PREPROCESSING

Handling duplicates

```
[ ]
# Checking duplicated values in dataset
count_duplicated = health_data.duplicated().sum()
print(f'Dataset having {count_duplicated} duplicated values')

Dataset having 0 duplicated values
```

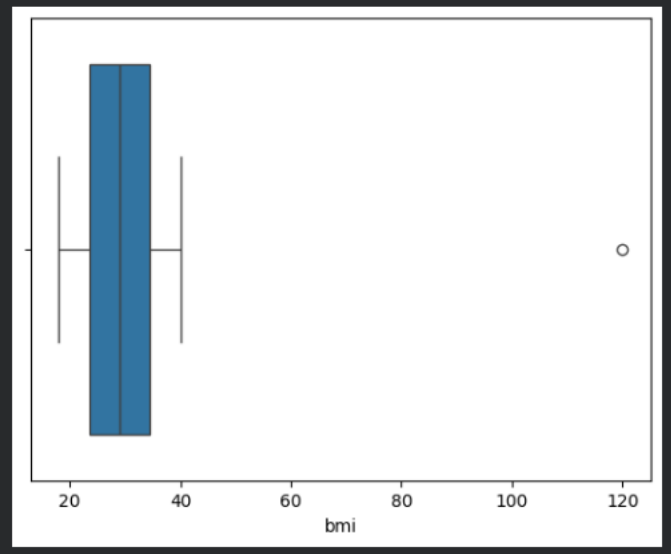
```
[ ]
# Check missing values before cleaning
print("Missing values before cleaning:")
print(health_data.isnull().sum())

# Fill missing numeric values with mean
health_data.fillna(health_data.mean(numeric_only=True), inplace=True)

# Fill missing categorical values with mode
health_data.fillna(health_data.mode().iloc[0], inplace=True)

# Check again after cleaning
print("\nMissing values after cleaning:")
print(health_data.isnull().sum())

Missing values before cleaning:
id                  0
age                 0
gender              0
bmi                 0
daily_steps         0
sleep_hours         0
water_intake_l      0
calories_consumed   0
smoker              0
alcohol             0
resting_hr          0
systolic_bp         0
diastolic_bp        0
cholesterol         0
family_history      0
disease_risk        0
dtype: int64
```

Handling missing values

OUTLIERS

```
# Add a BMI outlier to a specific row (example: row index 0)
health_data.loc[0, "bmi"] = 120
```

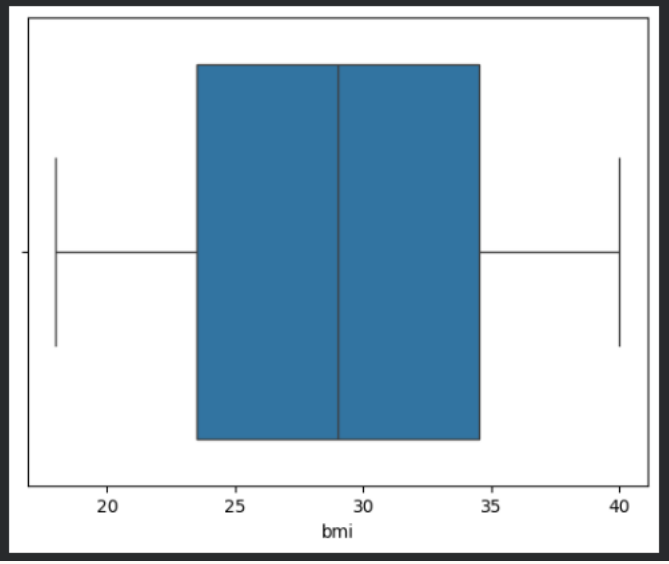
```
p1=sb.boxplot(x=health_data['bmi'])
```




```
outliers = health_data[(health_data['bmi'] > 50) | (health_data['disease_risk'] < 20)]
```

```
outliers =['bmi']
```

```
health_data2=health_data[(health_data['bmi']<=50)]
p1=sb.boxplot(x=health_data2['bmi'])
```



Detecting the outliers and dropping them

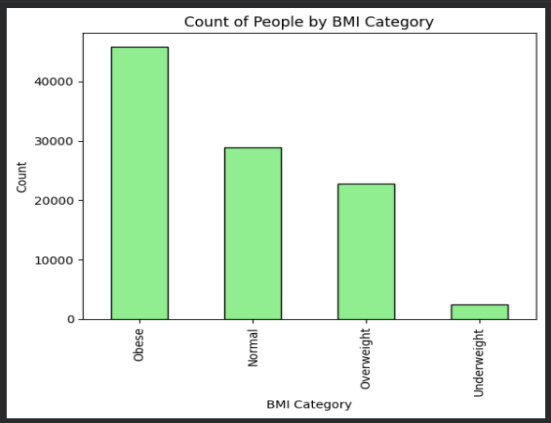
FEATURE ENGNEERING

-Adding a new calculated columns ‘BMI_Category’ and ‘Age_Category’
And visualizing them using a bar charts

```
# Add a new calculated column
health_data['BMI_Category'] = pd.cut(
health_data['bmi'],
bins=[0, 18.5, 24.9, 29.9, 100],
labels=['Underweight', 'Normal', 'Overweight', 'Obese']
)
health_data.head()
```

	id	age	gender	bmi	daily_steps	sleep_hours	water_intake_l	calories_consumed	smoker	alcohol	resting_hr	systolic_bp	diastolic_bp	cholesterol	family_history	disease_risk	WeeklyExercise	HealthScore	BMI_Category
0	1	56	Male	120.0	4198	3.9	3.4	1602	0	0	97	161	111	240	0	0	29386	-81.0	NaN
1	2	69	Female	33.3	14359	9.0	4.7	2346	0	1	68	116	65	207	0	0	100513	56.7	Obese
2	3	46	Male	31.6	1817	6.6	4.2	1643	0	1	90	123	99	296	0	0	12719	34.4	Obese
3	4	32	Female	38.2	15772	3.6	2.0	2460	0	0	71	165	95	175	0	0	110404	-2.2	Obese
4	5	60	Female	33.6	6037	3.8	4.0	3756	0	1	98	139	61	294	0	0	42259	4.4	Obese

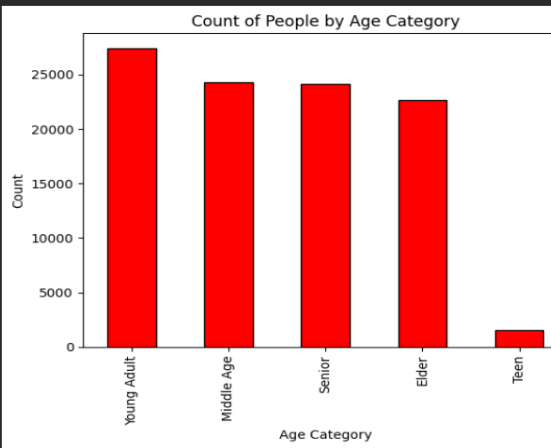
```
health_data['BMI_Category'].value_counts().plot(
    kind='bar', color='lightgreen', edgecolor='black'
)
plt.title('Count of People by BMI Category')
plt.xlabel('BMI Category')
plt.ylabel('Count')
plt.show()
```




```
# Add a new calculated column
health_data['Age_Category'] = pd.cut(
health_data['age'],
bins=[0, 18, 35, 50, 65, 100],
labels=['Teen','Young Adult' , 'Middle Age', 'Senior', 'Elder']
)
health_data.head()
```

	id	age	gender	bmi	daily_steps	sleep_hours	water_intake_l	calories_consumed	smoker	alcohol	resting_hr	systolic_bp	diastolic_bp	cholesterol	family_history	disease_risk	WeeklyExercise	HealthScore	BMI_Category	Age_Category
0	1	56	Male	120.0	4198	3.9	3.4	1602	0	0	97	161	111	240	0	0	29386	-81.0	NaN	Senior
1	2	69	Female	33.3	14359	9.0	4.7	2346	0	1	68	116	65	207	0	0	100513	56.7	Obese	Elder
2	3	46	Male	31.6	1817	6.6	4.2	1643	0	1	90	123	99	296	0	0	12719	34.4	Obese	Middle Age
3	4	32	Female	38.2	15772	3.6	2.0	2460	0	0	71	165	95	175	0	0	110404	-2.2	Obese	Young Adult
4	5	60	Female	33.6	6037	3.8	4.0	3756	0	1	98	139	61	294	0	0	42259	4.4	Obese	Senior

```
health_data['Age_Category'].value_counts().plot(
    kind='bar', color='red', edgecolor='black'
)
plt.title('Count of People by Age Category')
plt.xlabel('Age Category')
plt.ylabel('Count')
plt.show()
```



```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
health_data['gender'] = le.fit_transform(health_data['gender'])
import pandas as pd

# One-hot encode categorical columns
health_data = pd.get_dummies(health_data, drop_first=True)
```

This code converts categorical health variables into numerical format using label encoding and one-hot encoding so they can be used by machine-learning models.

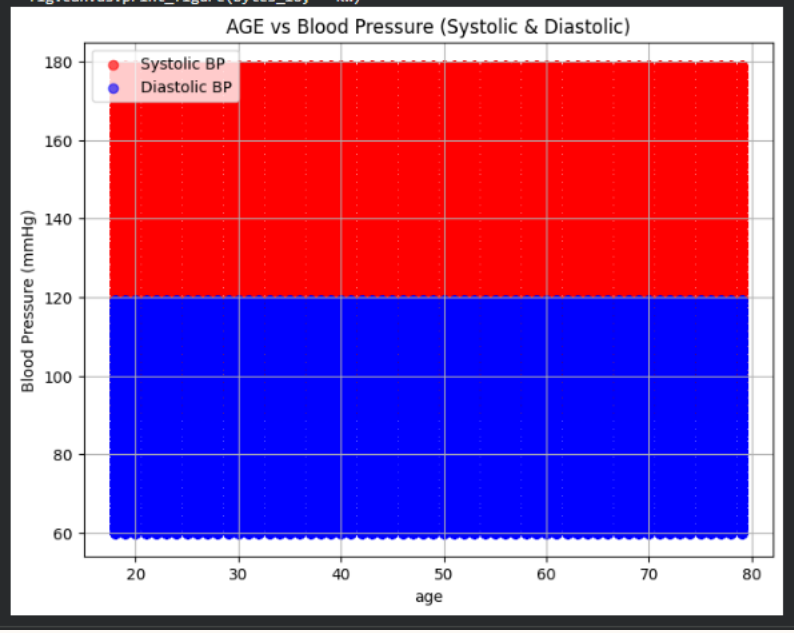
```
## Separating independant variables and dependant variable

# Creating the dataset with all dependent variables
dependent_variable = 'disease_risk'

# Creating the dataset with all independent variables
independent_variables = list(set(health_data.columns.tolist()) - {dependent_variable})

# Create the data of independent variables
X = health_data[independent_variables].copy()
# Create the data of dependent variable
y = health_data[dependent_variable].copy()
```

This code separates the dataset into input features (X) and the target variable (y) so a supervised machine-learning model can be trained to predict disease risk.

Data Visualization

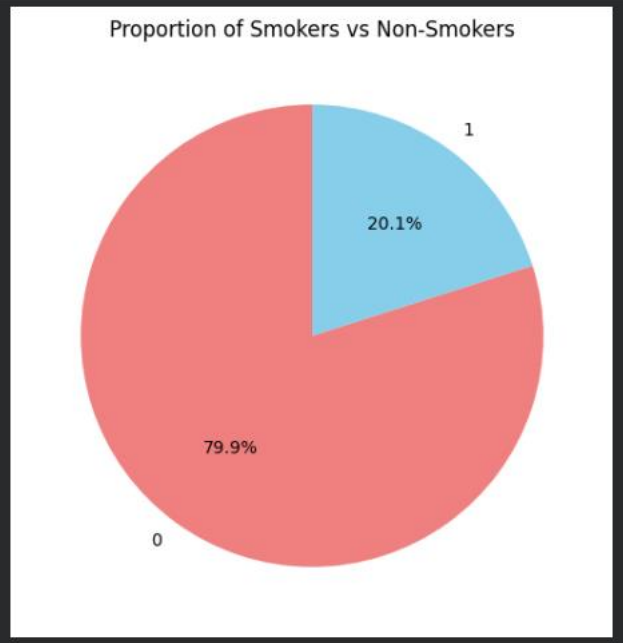

```
plt.figure(figsize=(8,6))
plt.scatter(health_data['age'], health_data['systolic_bp'], color='red', alpha=0.6, label='Systolic BP')
plt.scatter(health_data['age'], health_data['diastolic_bp'], color='blue', alpha=0.6, label='Diastolic BP')

plt.title('AGE vs Blood Pressure (Systolic & Diastolic)')
plt.xlabel('age')
plt.ylabel('Blood Pressure (mmHg)')
plt.legend()
plt.grid(True)
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Creating legend with loc="best" can be slow with large amounts of data.
  fig.canvas.print_figure(bytes_io, **kw)
```



This scatter plot visualizes how systolic and diastolic blood pressure vary with age, helping identify trends and potential health risks.

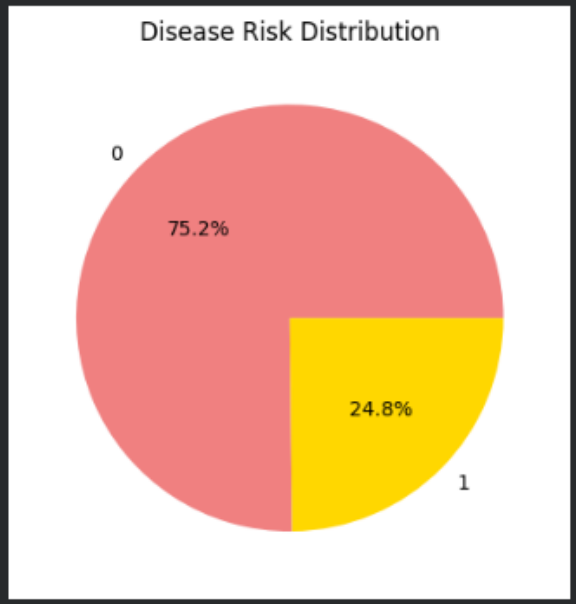
```
smoker_counts = health_data['smoker'].value_counts()

plt.figure(figsize=(6,6))
plt.pie(smoker_counts, labels=smoker_counts.index, autopct='%1.1f%%', startangle=90, colors=['lightcoral','skyblue'])
plt.title('Proportion of Smokers vs Non-Smokers')
plt.show()
```



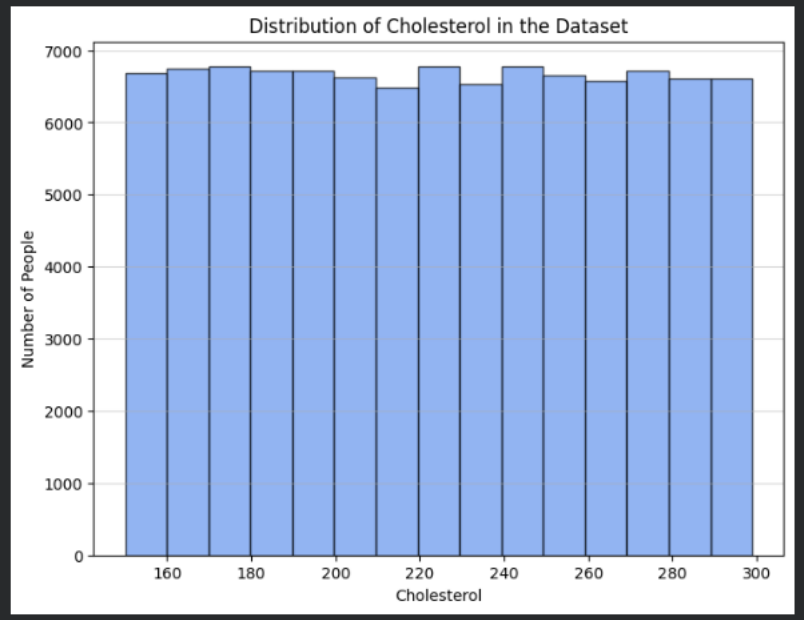
A pie chart that visualizes the proportion of smokers and non-smokers

This pie chart visualizes the distribution of disease risk categories, highlighting the proportion of individuals in each risk group.

```
health_data['disease_risk'].value_counts().plot(
    kind='pie', autopct='%1.1f%%', colors=['lightcoral', 'gold', 'lightblue']
)
plt.title('Disease Risk Distribution')
plt.ylabel('')  # hide y-label
plt.show()
```



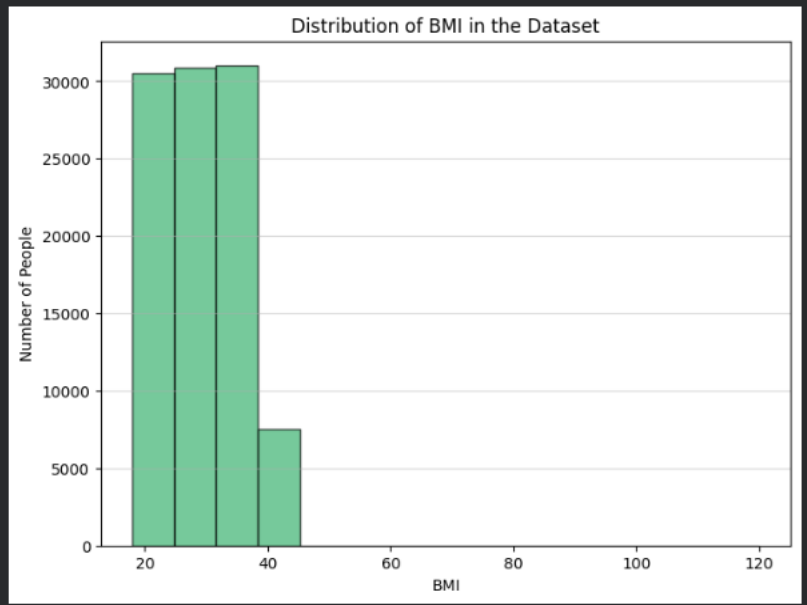
```
plt.figure(figsize=(8,6))
plt.hist(health_data['cholesterol'], bins=15, color='cornflowerblue', edgecolor='black', alpha=0.7)

plt.title('Distribution of Cholesterol in the Dataset')
plt.xlabel('Cholesterol')
plt.ylabel('Number of People')
plt.grid(axis='y', alpha=0.5)
plt.show()
```



This histogram shows the distribution of cholesterol levels in the dataset, helping identify patterns, spread, and potential outliers.

This histogram visualizes the distribution of BMI values to analyze spread, detect outliers, and understand population health patterns.

```
plt.figure(figsize=(8,6))
plt.hist(health_data['bmi'], bins=15, color='mediumseagreen', edgecolor='black', alpha=0.7)

plt.title('Distribution of BMI in the Dataset')
plt.xlabel('BMI')
plt.ylabel('Number of People')
plt.grid(axis='y', alpha=0.5)
plt.show()
```



CORRELATION

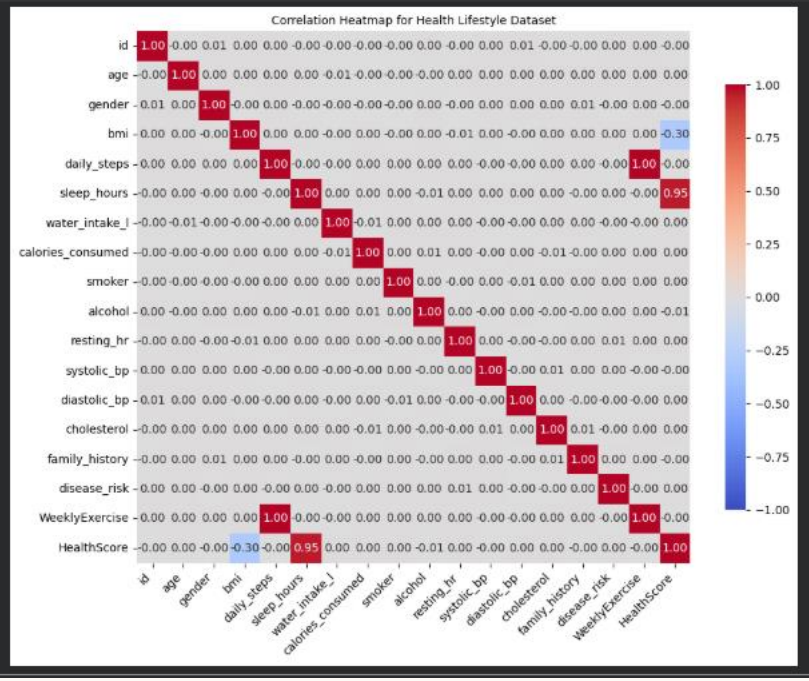
```
numeric_data = health_data.select_dtypes(include=['int64', 'float64'])
```

```
corr = numeric_data.corr()
corr
```

	id	age	gender	bmi	daily_steps	sleep_hours	water_intake_l	calories_consumed	smoker	alcohol	resting_hr	systolic_bp	diastolic_bp	cholesterol	family_history	disease_risk	WeeklyExercise	HealthScore
id	1.000000	-0.001392	0.005690	0.000950	0.001470	-0.000693	-0.000532	-0.004512	-0.001161	0.003285	-0.002270	0.001651	0.005161	-0.001686	-0.002013	0.000294	0.001470	-0.000946
age	-0.001392	1.000000	0.000124	0.001262	0.004847	0.000437	-0.005644	-0.002684	-0.002455	-0.000923	0.001460	0.000806	0.000178	0.000747	0.002651	0.001671	0.004847	0.000038
gender	0.005690	0.000124	1.000000	-0.000223	0.001245	-0.003120	-0.003173	-0.000580	-0.002921	0.004643	-0.001730	0.002609	0.000523	0.001240	0.005417	-0.001957	0.001245	-0.002910
bmi	0.000950	0.001262	-0.000223	1.000000	0.001633	0.000721	-0.003552	0.002066	-0.003481	0.002034	-0.006076	0.000964	-0.002903	0.002099	0.002164	0.003492	0.001633	-0.299395
daily_steps	0.001470	0.004847	0.001245	0.001633	1.000000	-0.001209	-0.000898	0.000306	0.000453	0.003456	0.004401	-0.002049	-0.003537	0.000123	0.002351	-0.003444	1.000000	-0.001644
sleep_hours	-0.000693	0.000437	-0.003120	0.000721	-0.001209	1.000000	0.001507	0.001229	0.001055	-0.006332	0.002628	0.000180	0.002243	0.003051	-0.001877	0.002582	-0.001209	0.953913
water_intake_l	-0.000532	-0.005644	-0.003173	-0.003552	-0.000898	0.001507	1.000000	-0.007590	0.001555	0.003664	0.000872	-0.003237	-0.001321	0.002576	0.001970	-0.001902	-0.000898	0.002504
calories_consumed	-0.004512	-0.002684	-0.000580	0.002066	0.000306	0.001229	-0.007590	1.000000	0.002223	0.005630	0.000889	-0.000171	0.004270	-0.005218	-0.004836	0.002737	0.000306	0.000553
smoker	-0.001161	-0.002455	-0.002921	-0.003481	0.000453	0.001055	0.001555	0.002223	1.000000	0.002472	-0.001439	0.003652	-0.007331	0.000999	0.002719	0.001125	0.000453	0.002051
alcohol	0.003285	-0.000923	0.004643	0.002034	0.003456	-0.006332	0.003664	0.005630	0.002472	1.000000	0.004570	-0.004618	0.003478	-0.003599	-0.002959	0.000515	0.003456	-0.006652
resting_hr	-0.002270	0.001460	-0.001730	-0.006076	0.004401	0.002628	0.000872	0.000889	-0.001439	0.004570	1.000000	0.000791	-0.001756	-0.001721	0.000108	0.005437	0.004401	0.004331
systolic_bp	0.001651	0.000806	0.002609	0.000964	-0.002049	0.000180	-0.003237	-0.000171	0.003652	-0.004618	0.000791	1.000000	-0.002257	0.006215	0.001134	0.001086	-0.002049	-0.000117
diastolic_bp	0.005161	0.000178	0.000523	-0.002903	-0.003537	0.002243	-0.001321	0.004270	-0.007331	0.003478	-0.001756	-0.002257	1.000000	0.001072	-0.001339	-0.003774	-0.003537	0.003011
cholesterol	-0.001686	0.000747	0.001240	0.002099	0.000123	0.003051	0.002576	-0.005218	0.000999	-0.003599	-0.001721	0.006215	0.001072	1.000000	0.005016	-0.003941	0.000123	0.002281
family_history	-0.002013	0.002651	0.005417	0.002164	0.002351	-0.001877	0.001970	-0.004836	0.002719	-0.002959	0.000108	0.001134	-0.001339	0.005016	1.000000	0.003225	0.002351	-0.002441
disease_risk	0.000294	0.001671	-0.001957	0.003492	-0.003444	0.002582	-0.001902	0.002737	0.001125	0.000515	0.005437	0.001086	-0.003774	-0.003941	0.003225	1.000000	-0.003444	0.001416
WeeklyExercise	0.001470	0.004847	0.001245	0.001633	1.000000	-0.001209	-0.000898	0.000306	0.000453	0.003456	0.004401	-0.002049	-0.003537	0.000123	0.002351	-0.003444	1.000000	-0.001644
HealthScore	-0.000946	0.000038	-0.002910	-0.299395	-0.001644	0.953913	0.002504	0.000553	0.002051	-0.006652	0.004331	-0.000117	0.003011	0.002281	-0.002441	0.001416	-0.001644	1.000000

This code extracts only the numeric columns from the dataset, preparing them for statistical analysis, visualization, or modeling, Then computes the correlation matrix of numeric features to examine linear relationships, identify multicollinearity, and inform feature selection.

```python
# Plot heatmap
plt.figure(figsize=(10, 8))
sb.heatmap(
    corr,
    annot=True,           # show numbers
    cmap='coolwarm',      # color map
    fmt='.2f',            # 2 decimals in numbers
    annot_kws={"size":10},
    vmin=-1,              # min value for color scale
    vmax=1,               # max value for color scale
    cbar_kws={'shrink':0.8}
)

plt.title("Correlation Heatmap for Health Lifestyle Dataset", fontsize=10)
plt.xticks(rotation=45, ha='right',fontsize=10)
plt.yticks(rotation=0,fontsize=10)
plt.tight_layout()
plt.show()
```



This heatmap visualizes correlations between numeric features, highlighting strong positive or negative relationships and aiding feature selection.

```
f_corr = corr[(corr >= 0.5) | (corr <= -0.5)]
print(f_corr)
```

```
                      id  age  gender  bmi  daily_steps  sleep_hours  \
id                   1.0  NaN     NaN  NaN          NaN          NaN
age                  NaN  1.0     NaN  NaN          NaN          NaN
gender               NaN  NaN     1.0  NaN          NaN          NaN
bmi                  NaN  NaN     NaN  1.0          NaN          NaN
daily_steps          NaN  NaN     NaN  NaN          1.0          NaN
sleep_hours          NaN  NaN     NaN  NaN          NaN     1.000000
water_intake_l       NaN  NaN     NaN  NaN          NaN          NaN
calories_consumed    NaN  NaN     NaN  NaN          NaN          NaN
smoker               NaN  NaN     NaN  NaN          NaN          NaN
alcohol              NaN  NaN     NaN  NaN          NaN          NaN
resting_hr           NaN  NaN     NaN  NaN          NaN          NaN
systolic_bp          NaN  NaN     NaN  NaN          NaN          NaN
diastolic_bp         NaN  NaN     NaN  NaN          NaN          NaN
cholesterol          NaN  NaN     NaN  NaN          NaN          NaN
family_history       NaN  NaN     NaN  NaN          NaN          NaN
disease_risk         NaN  NaN     NaN  NaN          NaN          NaN
WeeklyExercise       NaN  NaN     NaN  NaN          1.0          NaN
HealthScore          NaN  NaN     NaN  NaN          NaN     0.953913

                     water_intake_l  calories_consumed  smoker  alcohol  \
id                              NaN                NaN     NaN      NaN
age                             NaN                NaN     NaN      NaN
gender                          NaN                NaN     NaN      NaN
bmi                             NaN                NaN     NaN      NaN
daily_steps                     NaN                NaN     NaN      NaN
sleep_hours                     NaN                NaN     NaN      NaN
water_intake_l                  1.0                NaN     NaN      NaN
calories_consumed               NaN                1.0     NaN      NaN
smoker                          NaN                NaN     1.0      NaN
alcohol                         NaN                NaN     NaN      1.0
resting_hr                      NaN                NaN     NaN      NaN
systolic_bp                     NaN                NaN     NaN      NaN
diastolic_bp                    NaN                NaN     NaN      NaN
cholesterol                     NaN                NaN     NaN      NaN
family_history                  NaN                NaN     NaN      NaN
disease_risk                    NaN                NaN     NaN      NaN
WeeklyExercise                  NaN                NaN     NaN      NaN
HealthScore                     NaN                NaN     NaN      NaN
```

```
                     resting_hr  systolic_bp  diastolic_bp  cholesterol  \
id                          NaN          NaN           NaN          NaN
age                         NaN          NaN           NaN          NaN
gender                      NaN          NaN           NaN          NaN
bmi                         NaN          NaN           NaN          NaN
daily_steps                 NaN          NaN           NaN          NaN
sleep_hours                 NaN          NaN           NaN          NaN
water_intake_l              NaN          NaN           NaN          NaN
calories_consumed           NaN          NaN           NaN          NaN
smoker                      NaN          NaN           NaN          NaN
alcohol                     NaN          NaN           NaN          NaN
resting_hr                  1.0          NaN           NaN          NaN
systolic_bp                 NaN          1.0           NaN          NaN
diastolic_bp                NaN          NaN           1.0          NaN
cholesterol                 NaN          NaN           NaN          1.0
family_history              NaN          NaN           NaN          NaN
disease_risk                NaN          NaN           NaN          NaN
WeeklyExercise              NaN          NaN           NaN          NaN
HealthScore                 NaN          NaN           NaN          NaN

                     family_history  disease_risk  WeeklyExercise  HealthScore
id                              NaN           NaN             NaN          NaN
age                             NaN           NaN             NaN          NaN
gender                          NaN           NaN             NaN          NaN
bmi                             NaN           NaN             NaN          NaN
daily_steps                     NaN           NaN             1.0          NaN
sleep_hours                     NaN           NaN             NaN     0.953913
water_intake_l                  NaN           NaN             NaN          NaN
calories_consumed               NaN           NaN             NaN          NaN
smoker                          NaN           NaN             NaN          NaN
alcohol                         NaN           NaN             NaN          NaN
resting_hr                      NaN           NaN             NaN          NaN
systolic_bp                     NaN           NaN             NaN          NaN
diastolic_bp                    NaN           NaN             NaN          NaN
cholesterol                     NaN           NaN             NaN          NaN
family_history                  1.0           NaN             NaN          NaN
disease_risk                    NaN           1.0             NaN          NaN
WeeklyExercise                  NaN           NaN             1.0          NaN
HealthScore                     NaN           NaN             NaN     1.000000
```

This code filters the correlation matrix to show only strong positive or negative correlations, helping identify redundant features for feature selection.

CHI-SQAURE

```
from sklearn.feature_selection import chi2
from sklearn.preprocessing import LabelEncoder
data = health_data.copy()

cat_cols = ['gender', 'smoker', 'alcohol', 'family_history']

# Label encode categorical columns
le = LabelEncoder()
for col in cat_cols:
    data[col] = le.fit_transform(data[col])
```

This code label-encodes categorical features to numeric values so they can be analyzed using the chi-square test for feature selection.

```
X = data[cat_cols]           # categorical features
y = data['disease_risk']     # target

chi_scores, p_values = chi2(X, y)
chi_square_df = pd.DataFrame({
    'Feature': cat_cols,
    'Chi2_Score': chi_scores,
    'p_value': p_values
})
```

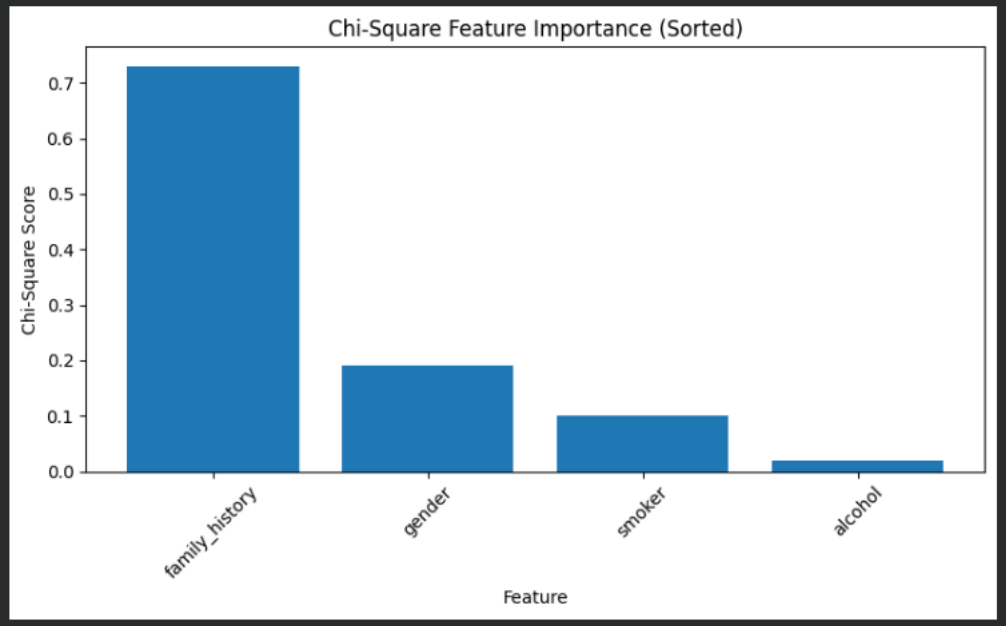
This code applies the chi-square test to categorical features to quantify their association with the target and help identify which features are most relevant for prediction.

```
# Sort by chi-square score (highest first)
chi_square_df = chi_square_df.sort_values(by='Chi2_Score', ascending=False)
print(chi_square_df)
```

```
          Feature  Chi2_Score   p_value
3  family_history    0.729134  0.393164
0          gender    0.190943  0.662133
1          smoker    0.101082  0.750536
2         alcohol    0.018597  0.891529
```

This code sorts categorical features by their chi-square scores, highlighting the strongest predictors for the target variable and aiding feature selection.

```python
# Visualization using matplotlib
plt.figure(figsize=(8,5))
plt.bar(chi_square_df['Feature'], chi_square_df['Chi2_Score'])
plt.xlabel("Feature")
plt.ylabel("Chi-Square Score")
plt.title("Chi-Square Feature Importance (Sorted)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



This bar chart visualizes chi-square feature importance scores, making it easy to identify which categorical features have the strongest association with the target variable.

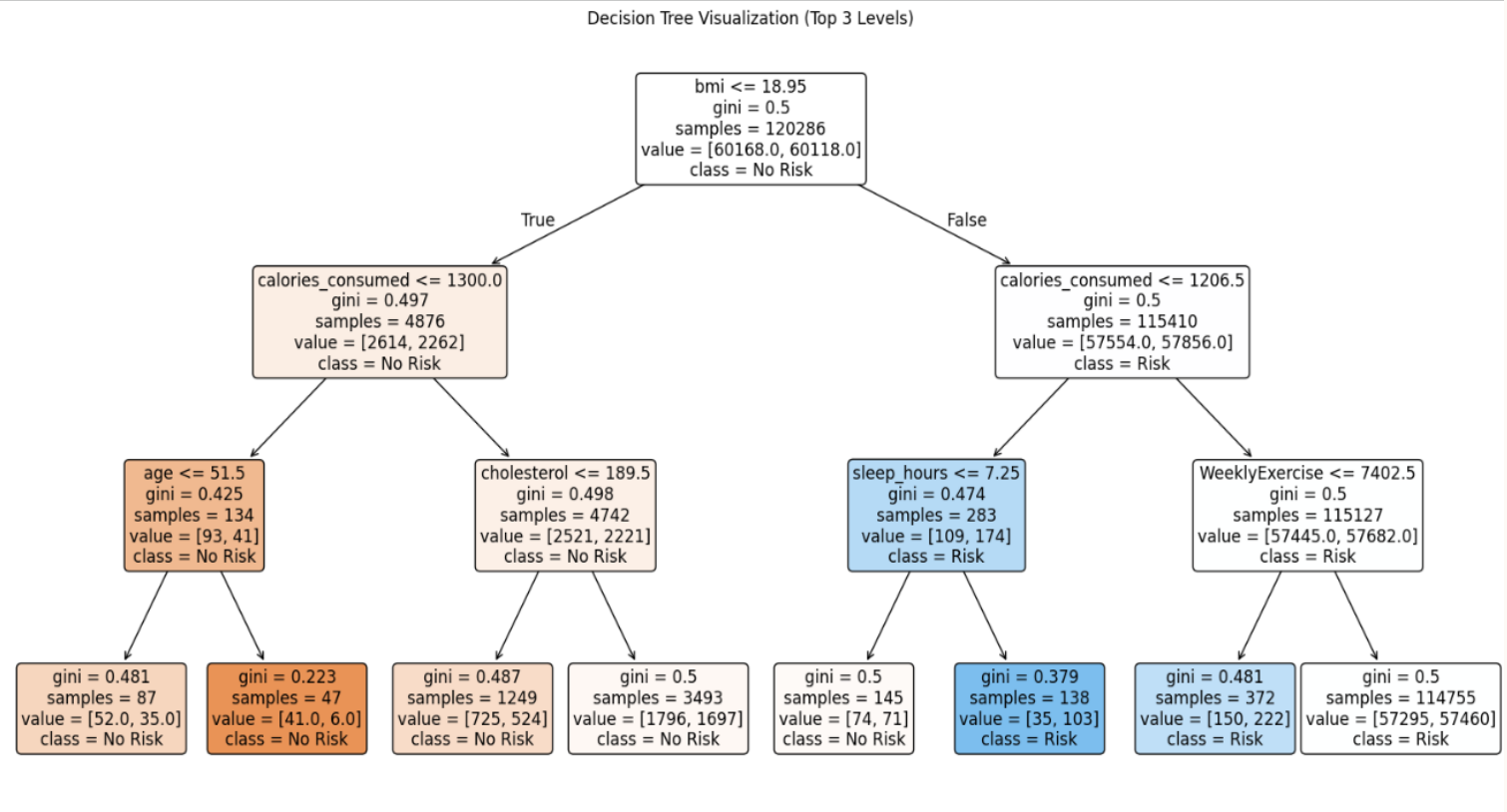
FEATURE SELECTION


```python
# ---------------------------
# 5. Select significant categorical features (or keep all if none)
# ---------------------------
alpha = 0.05
cat_selected = chi_square_df[chi_square_df['p_value'] < alpha]['Feature'].tolist()

if not cat_selected:
    print("\nNo categorical features are statistically significant. Keeping all categorical features for training.")
    cat_selected = cat_cols.copy()
else:
    print("\nSelected categorical features based on chi-square:")
print(cat_selected)

# ---------------------------
# 6. Combine with numerical features
# ---------------------------
# Ensure numerical columns are a list and exclude target and 'id'
num_cols = numeric_data.columns.tolist()
clean_num_cols = [col for col in num_cols if col not in ['id', 'disease_risk']]

# Combine numerical + categorical features, remove duplicates
final_selected_features = list(set(clean_num_cols + cat_selected))

print("\n============================")
print("FINAL SELECTED FEATURES")
print("============================")
print(final_selected_features)
```

```
No categorical features are statistically significant. Keeping all categorical features for training.
['gender', 'smoker', 'alcohol', 'family_history']

============================
FINAL SELECTED FEATURES
============================
['family_history', 'bmi', 'daily_steps', 'WeeklyExercise', 'alcohol', 'systolic_bp', 'water_intake_l', 'diastolic_bp', 'gender', 'sleep_hours', 'smoker', 'cholesterol', 'age', 'resting_hr', 'calories_consumed', 'HealthScore']
```

This code selects statistically significant categorical features using the chi-square test, combines them with relevant numerical features, and prepares the final feature set for model training.

DECISION TREE

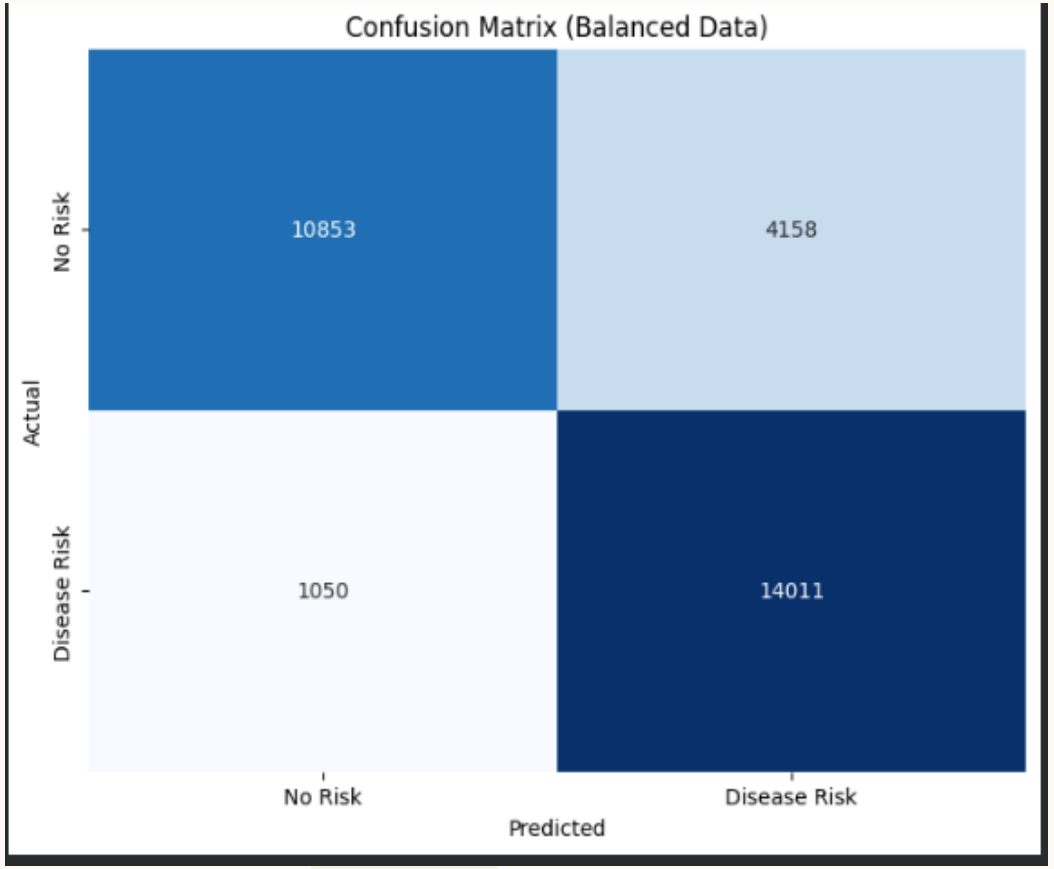


The decision tree indicates that BMI is the primary predictor of disease risk, with higher BMI combined with lifestyle factors such as calorie intake, sleep duration, and physical activity increasing the likelihood of disease.

	Accuracy	Precision	Recall	F1 Score
	83%			
0		91%	72%	81%
1		77%	93%	84%

CONFUSION MATRIX

```
from sklearn.metrics import confusion_matrix
from sklearn.preprocessing import LabelEncoder
from sklearn.utils import resample
# 8. Confusion Matrix
y_pred = clf.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

# 9. Plotting
plt.figure(figsize=(8, 6))
sb.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
           xticklabels=['No Risk', 'Disease Risk'],
           yticklabels=['No Risk', 'Disease Risk'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix (Balanced Data)')
plt.savefig('confusion_matrix_heatmap.png')

print("Confusion Matrix values:")
print(cm)
```



True Negatives (10853): Correctly predicted No Risk cases.

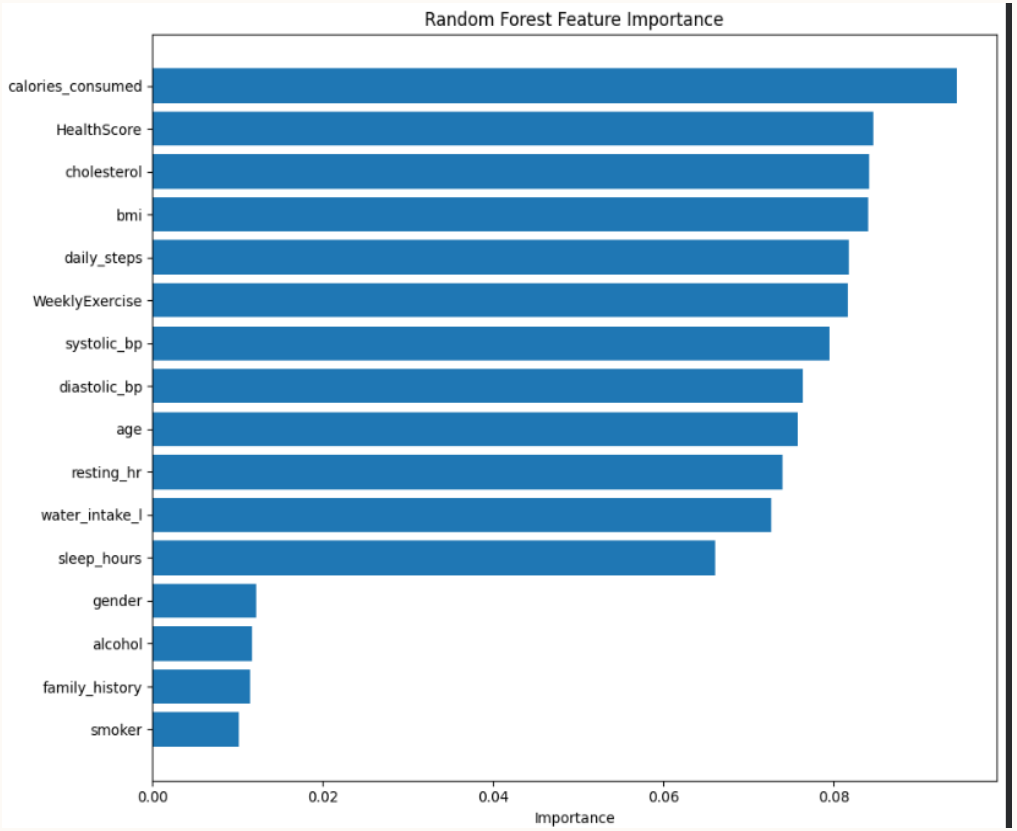
False Positives (4158): No-risk individuals incorrectly classified as Disease Risk.

False Negatives (1050): High-risk individuals incorrectly classified as No Risk.

True Positives (14011): Correctly predicted Disease Risk cases.

RANDOM FOREST

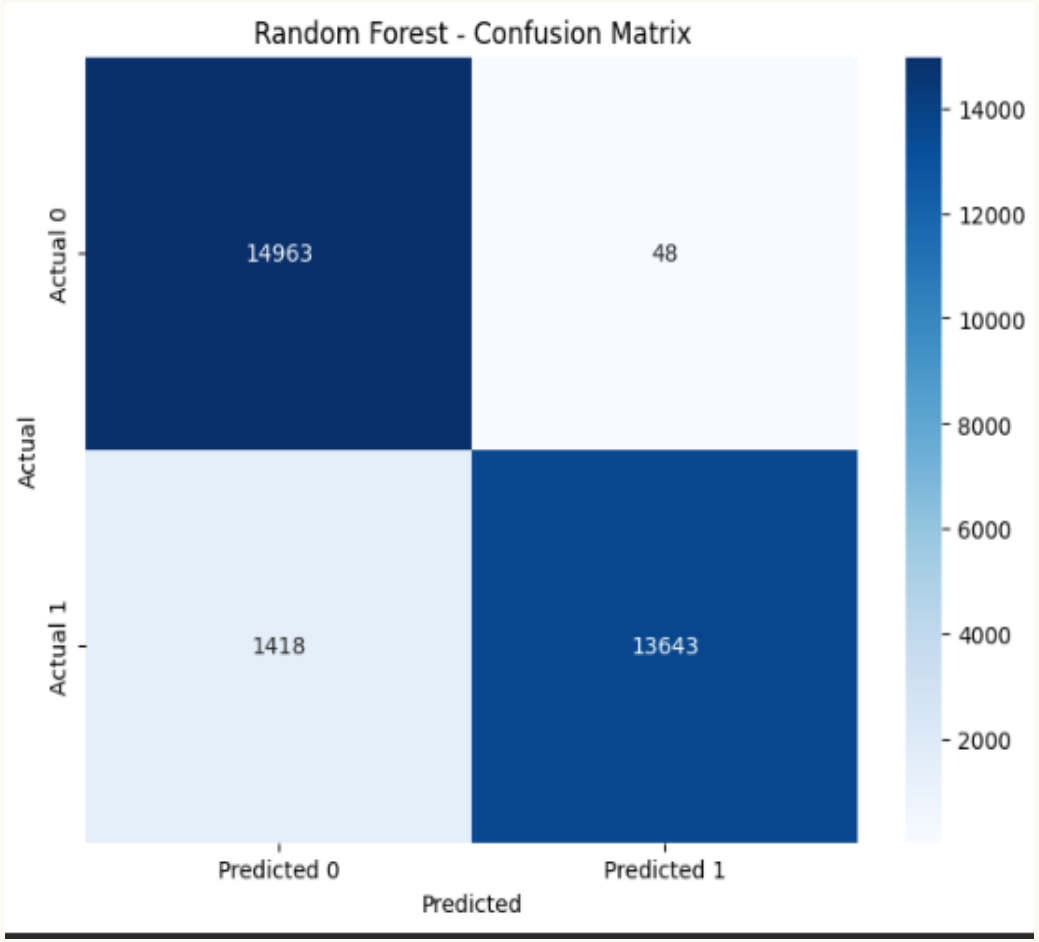
Random Forest was selected because it can handle multiple health and lifestyle features, capture complex relationships between them, and reduce overfitting by combining multiple decision trees. It provides more stable and accurate predictions than a single decision tree, making it suitable for disease risk prediction.



	Accuracy	Precision	Recall	F1 Score
	95%			
0		91%	100%	95%
1		100%	91%	95%

CONFUSION MATRIX

```python
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# 1. Generate predictions
y_pred = rf.predict(X_test)

# 2. Compute confusion matrix
cm = confusion_matrix(y_test, y_pred)

# 3. Plot heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])

plt.title("Random Forest - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



True Negatives (14963): Correct no-risk predictions

False Positives (48): Very few healthy individuals misclassified as at risk

False Negatives (1418): Some high-risk cases missed

True Positives (13643): Correct disease-risk predictions

NAIVE BAYES

	Accuracy	Precision	Recall	F1 Score
	51%			
0		75%	51%	61%
1		25%	49%	33%

Why Naive Bayes performs poorly here

- Strong feature correlations (e.g., BMI, blood pressure, cholesterol) violate the independence assumption.
- Health and lifestyle data are non-Gaussian and interdependent.
- The model cannot capture complex relationships in the data.

LOGISTIC REGRESSION

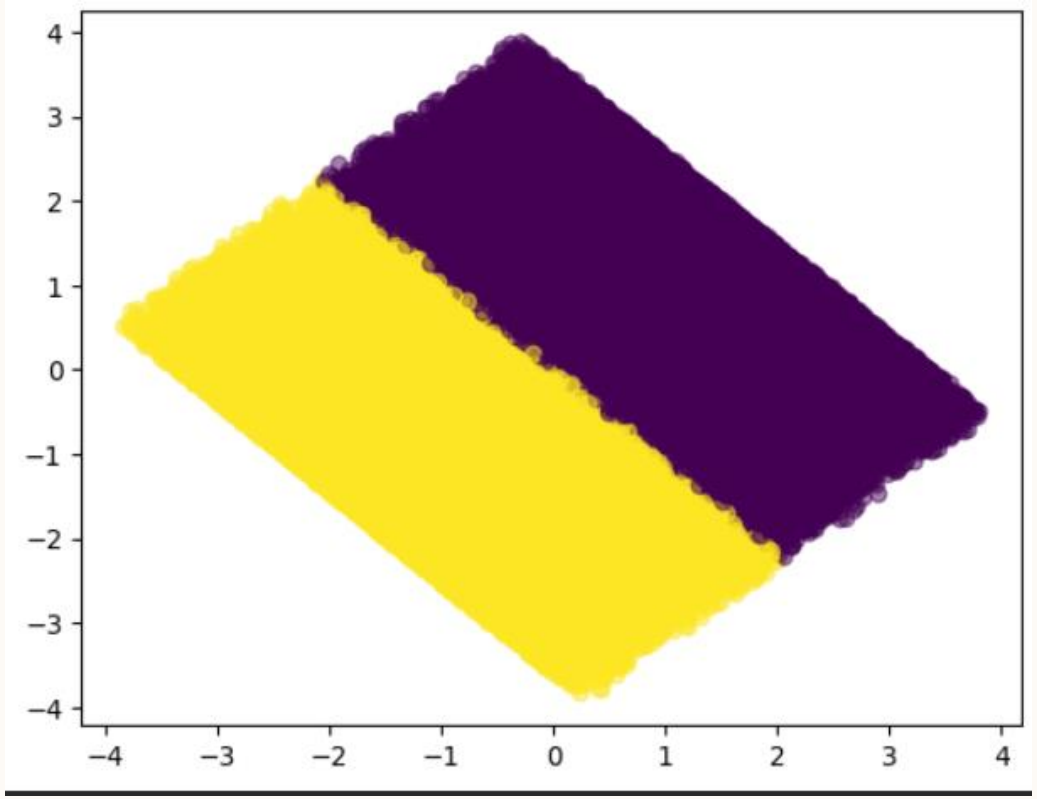
	Accuracy	Precision	Recall	F1 Score
	50%			
0		75%	51%	61%
1		25%	49%	33%

Justification

- It failed because your data is Non-Linear. You can't separate sick people from healthy people with a straight line (e.g., very high sleep is bad, very low sleep is bad, average is good—a line can't capture that).
- Pros:
- Math: Gives you exact numbers (e.g., "Smoking increases risk by 20%").
- Cons:
- Too Simple: Fails on complex patterns (like the "Donut Problem").
- Requires Scaling: Breaks if you don't scale your numbers (0 to 1).

K-MEANS CLUSTERING

K-Means clustering was applied to explore whether individuals naturally form distinct groups based on their lifestyle and health characteristics. Since it is an unsupervised method, it does not use the disease risk label, making it useful for discovering hidden structure in the data. The PCA visualization shows two clearly separated clusters, indicating that lifestyle behaviors strongly differentiate individuals. This suggests that the dataset contains meaningful patterns—likely representing 'healthier' versus 'less healthy' lifestyles—which supports the reliability of the features used for subsequent supervised learning models.



"The PCA visualization shows two clearly separated clusters, indicating that the health and lifestyle features naturally divide individuals into two distinct groups with minimal overlap, demonstrating strong cluster structure in the dataset."

THANK YOU